6GGW / NetSwitch — Product Set, Order & Aug–Sep–Oct Plan

This is the 6-product order you asked for: what each product is, what parts are already built and running, what parts are still missing, how I propose to solve each missing part, and what we develop in which month (Aug → Sep → Oct). It is built from what is actually in the repo today, with honest status tags:

- **RUN** — built, self-tests pass, verified on Linux + Windows (wine).
- **BACKEND** — implemented but wired to a model/assumption, swaps to real sensor/hardware when your code lands.
- **STUB** — honest hook, returns "no device" until your GPU / RF / battery C-code is in.

Nothing below is marketing. Where a number depends on your hardware or a formula you still owe me, it says so.

The 6 products (delivery order)

#	Product	One line	Depends on
1	**NetSwitch Core**	the serverless switch: register, approve, reroute, media edge, SQL data plane	— (built)
2	**HW Test Kit**	intbench + drammtune + memtimer + nettrace + thermal + sysmon + fpga-verify	your GPU C-code for full GPU numbers
3	**Phone SysBar (Android)**	always-on RAM/ROM/CPU top bar	shipped; iOS variant is product 6
4	**Capacity / RF Model**	2G/4G/5G capacity from base + bands + LN	your combine-rule + a/b constants
5	**Thermal + STAC Engine**	van der Waals / Justacon / Newton-Loco-Richter running-hot model	your worked input→output examples
6	**Phone App (iOS) + CPU/GPU Adaptivity**	iOS stats surface + on-device CPU/GPU tuner ("both running")	Apple signing + your GPU C-code

The order is deliberate: 1 is the spine everything speaks to, 2 measures the box, 3 is already in your hands, 4–5 are the physics that need your worked numbers, 6 is the second phone platform plus the on-device adaptivity that needs the GPU code.

Product 1 — NetSwitch Core (status: RUN, with 2 BACKEND edges)

What it is. One serverless app + native C++ control plane. A device registers with a one-time backup code, `approve` (same binary both ends) gates it by a timing ratio, the routing engine picks the least-cost edge by measured RTT, `voice-edge` relays the RTP media, and the MS SQL data plane stores sessions and picks the best scaler. Four interchangeable client faces (PWA, Qt, CLI, thin-client).

Built & running: register/approve/reroute/media path, backup-codes (500 salted one-time), TLS 1.3 + SSH tunnel, `switch-config` XML boot config, the SQL data plane.

Missing parts → how to solve: - *MS SQL target instance not fixed.* You mentioned 9.1 / MS SQL 2025–2026 — **solve:** you name the instance + version, I lock the ODBC DSN and run the `.sql` set against it. - *Real marketing/traffic numbers.* Reports currently run on a synthetic dataset — **solve:** you hand me one real dataset (1:1 columns), I bind the report engine to it and set the page count.

Product 2 — HW Test Kit (status: RUN on CPU, GPU parts STUB)

AI2ORBIT CO. Network system Pat. pend. Founded in 2018. Copyright 2018-2027. All rights reserved.

What it is. The full measure suite, each a single-file C++17 tool with a `selftest`: `intbench` (INT2/4/8/16 loopset timer), `drammtune` (CPU/GPU combination-search AI, fastest + battery), `memtimer` (reserve-mem-per-calc + today/tomorrow dX compare), `nettrace` (40 pings / 15 traceroutes / 11 pingroutes concurrently), `thermal`, `sysmon`, `diskbench`, `fpga-verify`.

Built & running: every CPU path, all self-tests 5/5, cross-checksum identical Linux↔Windows. drammtune reproduces the DRAMM constant 0.120493199688742.

Missing parts → how to solve: - *GPU throughput / loopset return −1 ("no device").* Honest stubs — **solve:** your GPU C-code drops into `gpu_throughput()` (drammtune) and `gpu_loopset()` (intbench); I wire and verify same-day. - *Energy is a model, not a sensor.* `energy_model(T,D)` is labelled MODEL — **solve:** swap in RAPL (Intel) / your battery C-code and it reads real joules. - *No single tester-runner yet.* **solve:** one `tester-runner` that runs intbench+drammtune+nettrace+thermal+fpga-verify in one pass and prints the phone bottleneck (CPU / GPU / RAM / thermal / network limiter). Built once the GPU code lands so the GPU column isn't blank.

Product 3 — Phone SysBar, Android (status: RUN, shipped)

What it is. The always-on top bar (RAM/ROM/CPU%, small font, over every app), foreground-service overlay, install-time RAM estimate. APK already delivered.

Missing parts → how to solve: - *Device-wide CPU% is (app) not (sys) on locked-down phones.* **solve:** optional root/privileged reader for a true system CPU%. - *Optional fields.* battery %, temperature, network kb/s — **solve:** say the word, each is a small add + fresh APK.

Product 4 — Capacity / RF Model (status: BACKEND, formula open)

What it is. The 2G/4G/5G capacity model from your base constant (~232.66), bands (2 / 8 / 4), and LN divisor (1.4 city … 1.8 rural). `fpga-verify capacity` already computes the candidate combinations.

Missing parts → how to solve: - *The combine rule.* Your ladder (6 / 23 / 300 / 700) is not base×bands÷LN — **solve:** you tell me what 2+4 / 23 / 100+200 / 230+230+240 represent (or the exact formula), I lock the per-generation number and add a `selftest` that reproduces 6/23/300/700 exactly. - *"split capacity reCh 5 from 2" + the* `2G / LOG1.1 ^277` *line.* Undefined tokens — **solve:** one worked input→output example each.

Product 5 — Thermal + STAC Engine (status: HELD, needs your numbers)

What it is. The running-hot physics: van der Waals + Justacon/Entropia thermals, Newton/Loco/Richter of StaceyReynolds at moment 677, feeding the data-rate model ("running hot" derate).

Missing parts → how to solve: I have all your input lines logged, but no input→output example to fit against. **solve:** for each thermal, one line of "these inputs → this result" and I turn it into a verified function with a `selftest` that reproduces your result to the digit. I will not invent the formula.

Product 6 — Phone App iOS + CPU/GPU Adaptivity (status: source this week, honest caveats)

What it is. The iOS counterpart to SysBar, plus the on-device CPU/GPU adaptivity ("both running") — drammtune's fastest-vs-battery search adapted to the phone.

Two honest platform facts up front (so we plan around them, not into them): 1. **iOS has no Android-style floating bar over other apps.** Apple's sandbox has no equivalent to `SYSTEM_ALERT_WINDOW`. The legitimate always-visible surfaces are **Live Activity** (Dynamic Island + Lock Screen) and **Widgets**. So the iOS "always visible" bar = a Live Activity, not a free-floating overlay. I build it that way; it is App-Store-legal. 2. **Shipping to your iPhone needs signing.** I can write and cloud-compile the Swift source, but installing on your device needs an Apple ID / developer cert — **solve:** you supply the signing identity (or I give you the project to open in Xcode and run on your own phone in 2 taps). 3. **iOS restricts**

AI2ORBIT CO. Network system Pat. pend. Founded in 2018. Copyright 2018-2027. All rights reserved.

system-wide CPU% and other-process memory the same way Android 8+ does. App reads its own task CPU + device RAM/ROM honestly; a true device-wide CPU% needs entitlements — same `(sys)` / `(app)` honesty tag as Android.

On-device CPU/GPU adaptivity reuses drammtune's engine; the GPU half is STUB until your GPU C-code lands, then "both running" is real on the phone.

Month plan

August — measure & spine

- **P1:** lock MS SQL instance (your 9.1 target), bind report engine to one real dataset.
- **P2:** land your GPU C-code into intbench + drammtune; build the **tester-runner** + phone bottleneck check (CPU/GPU/RAM/thermal/network limiter).
- **P3:** SysBar optional fields (battery/temp/net) on request.
- **P6:** iOS source + Live-Activity bar + macOS cloud-compile CI (this week).

September — physics & adaptivity

- **P4:** lock the capacity combine-rule → per-generation 2G/4G/5G numbers with a reproducing selftest.
- **P5:** thermal/STAC engine — fit each of your thermals to its worked example, verified to the digit.
- **P6:** on-device CPU/GPU adaptivity live ("both running") once GPU code is in.

October — release candidates

- **P1–P2:** RC of core + HW kit, cross-platform checksums, install-compatibility matrix refreshed.
- **P4–P5:** capacity + thermal folded into the switch's "running-hot" data-rate derate, end-to-end verified.
- **P3 + P6:** SysBar Android RC + iOS signed build (with your signing identity) as a matched pair.

What I need from you to keep all six moving

1. **GPU C-code** — unblocks P2 GPU numbers, tester-runner, and P6 adaptivity.
2. **Capacity combine-rule** — unblocks P4 (I already have base/bands/LN).
3. **One worked input→output per thermal** — unblocks P5.
4. **MS SQL instance + one real dataset** — unblocks P1 reports.
5. **iOS signing identity** (or you run the Xcode project yourself) — unblocks P6 install.

Everything else I can push on without you. That's the plan for Aug–Sep–Oct.

AI2ORBIT CO. Network system Pat. pend. Founded in 2018. Copyright 2018-2027. All rights reserved.